COLLABORATIVE FAULT TOLERANCE IN FOG COMPUTING USING REVERSE AUCTION AND GOSSIP

Abdus Saboor Gaffari Mohammed
Department of Computer Science and Engineering
College of Engineering
American University of Sharjah
Sharjah, UAE
b00105302@aus.edu

Solomon Ghebretatios
Department of Computer Science and Engineering
College of Engineering
American University of Sharjah
Sharjah, UAE
b00104390@aus.edu

Affan Murtaza
Department of Computer Science and Engineering
College of Engineering
American University of Sharjah
Sharjah, UAE
b00104949@aus.edu

December 2, 2025

ABSTRACT

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

Keywords fog computing · fault tolerance · collaborative · reverse auction · gossip · trust

1 Introduction

Internet of Things (IoT) systems have moved from small prototypes to infrastructure that entire cities and businesses depend on. Smart traffic lights, industrial plants, retail analytics, energy grids, and even campus security all rely on streams of sensor data that must be processed in near real time. Sending everything to a distant cloud data center is often too slow and too expensive in terms of bandwidth, and it can expose critical services to wide-area network failures. This has driven the rise of edge and fog computing, where at least part of the computation is brought closer to where data is produced. Fog computing in particular introduces a distributed layer of compute, storage, and networking between edge devices and central clouds, hosted on gateways, micro data centers, base stations, or other intermediate nodes.[**?**]

In a typical edge-fog-cloud hierarchy, resource-constrained edge devices such as cameras, sensors, or embedded controllers generate tasks that cannot be fully processed locally. These tasks are offloaded to more powerful fog nodes, which aggregate and process data from many nearby devices, and then optionally forward summaries or heavier workloads to the cloud for deeper analytics or long term storage. This layered model can significantly reduce end-to-end latency and wide-area traffic compared to cloud-only deployments, while still benefiting from the elasticity and global visibility of the cloud[**?**].

However, gaining these benefits comes with a price. Fog nodes typically do not have the scale, redundancy, or environmental control of large cloud data centers. They are deployed in diverse locations, may be owned by different organizations, and are often shared across multiple applications[**?**]. As a result they are exposed to a wide range of

faults: hardware failures, resource exhaustion under bursty demand, performance degradation, and network issues that can isolate parts of the hierarchy. Since individual fog nodes host many latency sensitive tasks, even a single node failure or severe slowdown can cascade into missed deadlines, poor user experience, or outright service outages [**?**].

Fault tolerance (FT) is therefore a central requirement for realistic fog deployments. In distributed and cloud systems, FT is commonly divided into reactive and proactive techniques. Reactive FT focuses on detecting a fault after it happens and recovering from it, for example by restarting a failed component, rolling back to a checkpoint, or failing over to another server. Proactive FT attempts to anticipate faults and act before they affect service, for example by monitoring indicators of impending failure and moving work away from risky nodes in advance [**?**]. Reactive approaches are often simpler and do not require prediction, but they can still allow short outages, lost progress, or SLA violations. Proactive approaches can reduce downtime and improve availability, but they depend on reasonably accurate prediction and careful coordination to avoid unnecessary overhead. In practice, robust systems tend to rely on a combination of both styles [**?**].

Container technologies make proactive and reactive FT particularly attractive in fog environments. Containers encapsulate an application and its dependencies in a lightweight, portable unit that can be started, paused, checkpointed, and resumed on different servers with relatively low overhead compared to full virtual machines. Modern checkpoint and restore tools allow the in-memory state of a running container to be captured and migrated, which enables live or near-live relocation of work when a node is predicted to fail or when it becomes overloaded [**?**]. For IoT workloads that need to keep running close to the edge, container-based migration is a natural building block for FT.

Beyond technical mechanisms, fog computing also has an important organizational dimension. Unlike a single cloud data center controlled by one provider, a wide-area fog deployment is often multi-owner. A telecom operator may host fog nodes at base stations, a municipality may deploy fog resources for smart city services, and private companies or campuses may add their own edge infrastructure. These different parties may agree to collaborate, but they do not form a single centrally controlled pool. Each fog node has its own local policies and economic incentives. A node might be willing to host additional containers for a neighbor if it has spare capacity and receives some compensation, but it should be able to decline if doing so would jeopardize its own SLAs or violate policy.

This multi-owner setting changes how FT can be designed. Many existing FT schemes for cloud or fog assume a central scheduler with an accurate global view that can simply assign or reassign tasks wherever it sees fit. Other work focuses on technical migration triggers or scheduling heuristics without considering that nodes might strategically misreport their available resources or migration costs if there is a benefit in doing so. As a result, there is a gap between the algorithms described in much of the FT literature and the behavior we should expect in a realistic, economically motivated fog ecosystem.

Across these works, a couple of important gaps stand out. Current container based fault tolerance mechanisms for fog computing still lean heavily on centralized schedulers or brokers that decide where containers should move, as in [**?**, **?**]. In practice, this can leave individual fog nodes with little say in whether they accept migrated workloads, even when their local conditions or policies suggest they should decline. At the same time, these approaches treat migration largely as a control decision made from the top, rather than as a collaborative process in which independently owned fog nodes express their preferences, constraints, and costs. As a result, the literature offers limited support for decentralized, locally informed migration decisions where fog nodes can negotiate, accept, or reject incoming containers based on their real time state, which is exactly the sort of autonomy we would expect in realistic multi owner fog deployments.

Our work aims to bridge this gap by viewing FT in fog computing as a collaborative, market-like process rather than a centrally imposed one. We consider a three layer edge-fog-cloud architecture where tasks from stationary edge devices are packaged into containers and executed primarily at the fog layer. An external prediction module (for example, a machine learning model trained on performance and health metrics) flags servers that are likely to experience faults in the near future. When such a prediction arrives, the system treats all containers on the affected server as at risk and seeks to relocate them before the fault materializes. If the prediction is wrong or a fault occurs unexpectedly, the same machinery can be used in a reactive mode to migrate or restart containers after the fact.

Instead of assuming that a central controller can dictate where these containers move, we design the decision process as a collaboration between fog nodes. Potential destination nodes receive information about containers that need to be moved and can place bids indicating the compensation they require and the conditions under which they can host the work. A reverse auction mechanism then selects winners, favoring nodes that offer a good balance of resource availability, cost, and historical reliability. At the same time, a lightweight gossip protocol spreads summarized load and reputation information across the fog layer so that nodes do not need a perfectly accurate global view but still have a reasonably up to date sense of the health of their neighbors. Over time, nodes that misreport their capabilities or frequently violate SLAs see their trust scores and chances of winning auctions decrease, while honest, dependable nodes are rewarded with more opportunities and greater revenue.

From a broader perspective, this approach treats FT in fog computing as both a technical and a socio-economic problem. Technically, we must move containers quickly enough, often enough, and to appropriate destinations so that services stay within their latency and reliability targets. Economically, we must design incentives so that independently administered fog nodes actually want to participate in the FT process, report their local state truthfully, and cooperate even when they are not under direct control of a single authority. By combining container-based migration, proactive prediction, reverse auctions, and gossip, our framework aims to provide a realistic path toward FT that respects the autonomy of each node while still behaving like a coordinated system from the perspective of applications.

The contributions made by this work are summarized as follows:

1. We design a three layer collaborative framework that spans edge, fog, and cloud. Edge devices generate computational tasks, the fog layer hosts them inside containerized environments, and the cloud serves as a backup and a point for large scale coordination. Within this hierarchy, fog nodes are able to communicate and cooperate when one node faces a fault or heavy load, so that services can keep running smoothly even when part of the infrastructure is under stress.
2. We enable flexible migration strategies that support container movement within a single fog node (intra fog), across neighbouring fog nodes (inter fog), and from the fog layer to the cloud when local resources are no longer sufficient. Migration decisions take into account available resources, predicted faults, and current network conditions. This adaptability helps maintain continuous and reliable service execution under changing workload and failure patterns.
3. We introduce distributed decision making at the fog layer. Each fog node monitors the health and utilization of its own servers and collaborates with nearby nodes when selecting migration targets. This removes the need for a single centralized controller, allows nodes to react more quickly to local events, and improves reliability during fault recovery by avoiding a single point of control.
4. We propose a collaborative design that keeps most of the control logic and negotiation close to the fog layer, which helps reduce delay and network overhead. Even if some servers fail or become unreliable, other fog nodes can step in, accept migrated containers, and restore service with minimal disruption. In this way, the framework aims to offer fault tolerance that better matches the latency, resilience, and autonomy requirements of real world fog computing deployments.

2 Related Work

Fault tolerance in distributed IoT execution environments has been approached from several directions, including container migration, proactive failure prediction, and fault-aware scheduling at the fog and edge layers. Together, these works emphasize the importance of maintaining service continuity under dynamic and resource-constrained conditions, while also revealing that integrated migration-centric mechanisms remain relatively under-explored.

Early works on edge–cloud container migration have focused primarily on improving migration decision-making. [**?**] proposed a fuzzy logic-based container migration mechanism for IoT-driven edge–cloud systems. The study considers CPU, memory, and network conditions to guide migration decisions, highlighting the value of adaptive, multi-metric decision models for reliability in heterogeneous environments. The framework relies on container runtimes that support CRIU (Checkpoint/Restore In Userspace), a widely used Linux utility for capturing a container's in-memory state and restoring it elsewhere.However, The study proposes the conceptual migrations process and lacks end-to-end implementation.

A more comprehensive container-level mechanisms is proposed in other Kubernetes-focused research which aim to incorporate off-the-shelf readily available technologies. [**?**] introduces a proactive architecture that combines BiLSTM based fault prediction with a stateful migration mechanism integrated into the Kubernetes control plane. Their use of CRIU enables transferring in-memory boot and execution states and demonstrates how stateful recovery can reduce startup delays and improve availability. This work establishes the feasibility of performing full state transfer for containerized services and shows its utility for maintaining service continuity.

Within fog computing, fault tolerance has mostly been considered at the task or module scheduling level. [**?**] proposed a Markov-based reliability model that dynamically allocates workloads to stable fog nodes. [**?**] similarly integrate reliability considerations into a hybrid scheduling approach that improves load distribution and success rates for latency-sensitive IoT tasks. These approaches offer useful models for assessing fog-layer reliability, although their mitigation actions occur through task reassignment rather than full service migration.

The work in [**?**] proposed the ECLB framework, which applies Bayesian classification and checkpointing to manage load and improve reliability in fog-based IoT applications. Their proposed model incorporates energy-aware backup

placement and dynamic balancing of modules across fog devices. The framework illustrates how lightweight statistical methods can strengthen task continuity. Similarly authors in [**?**] also proposes a task based decentralized allocation strategy with the aim of achieving resilience through group formation and local cooperation during node failures.

The authors in [**?**] proposed two complementary frameworks for decentralized task offloading in vehicular edge environments. In the first framework, called the Autonomous Vehicular Edge (AVE), each vehicle gathers lightweight beacons from nearby peers and applies an ant colony optimization strategy to offload tasks based on neighbor availability, link quality, and mobility characteristics. The second framework, Hybrid Vehicular Cloud (HVC), extends this model by incorporating roadside units and cloud servers, and uses an online scheduler to decide dynamically whether a task should be executed by a neighboring vehicle, an RSU, or the cloud. Both frameworks demonstrate the benefits of decentralized decision-making and multi-tier offloading paths for reducing latency and maintaining high availability.

Moreover, surveys such as [**?**] have attempted to summarize architectural and scheduling techniques ranging from reinforcement learning to stochastic optimization, which are used to improve fault tolerance in fog computing systems. These reviews highlight the constant challenges related to dynamic systems, heterogeneity of resources, and real-time reliability in fog environments which point toward the need for unified solution solutions that offer orchestration and recovery across distributed compute tiers.

Other works have also explored mechanisms for enforcing truthfulness in collaborative distributed computing paradigms. [**?**] explores reverse-auction–based resource allocation models in cloud settings. In their work, users bid for computational resources and the system allocates them based on fair pricing and optimal social welfare. The framework attempts to ensure allocation outcomes remain both efficient and economically balanced. While it does not consider fault tolerance, the study underscores the relevance of fairness, cost-awareness, and multi-tenant optimization for guiding allocation and migration decisions. Lastly, [**?**] proposed an online auction-based incentive mechanism for Collaborative Edge Computing, where edge servers independently decide whether to accept incoming tasks based on dynamic virtual pricing of CPU, storage, and bandwidth. The framework uses primal–dual optimization to handle stochastic task arrivals, soft deadlines, and heterogeneous valuations while ensuring truthfulness and competitive social welfare.The study illustrates how decentralized edge servers can make autonomous, locally informed decisions in distributed environments.

Across these works, several gaps remain evident. Existing fog and container-fault-tolerance mechanisms often rely on centralized schedulers, where migration or task-placement decisions are made by a single broker or cluster manager as seen in [**?**][**?**][**?**][**?**]. These centralized designs can unintentionally force fog nodes to accept migrated tasks or containers, even when local conditions make them unsuitable. Moreover, many fault-tolerant fog approaches operate strictly at the task level rather than supporting stateful container mobility, while predictive mechanisms such as those used by [**?**] and [**?**] do not integrate migration execution into a single, continuous workflow. Taken together, the literature shows limited support for decentralized, locally informed migration decisions in which fog nodes themselves retain control over accepting, rejecting, or proposing alternative placement options based on real-time resource conditions. This is especially crucial in a multi-owner environment such as that proposed in our approach. Our study addresses these gaps by designing a proactive fault-tolerance mechanism in which each fog node participates directly in a collaborative decentralized decision-making, and evaluates migration candidacy using its own state, neighbourhood conditions, and partial-view system context. We aim to design a framework which avoids rigid top-down placement policies, allowing nodes to maintain autonomy, and enable container migration to proceed in a manner that better reflects the volatility, dynamic agent behaviour, and resource heterogeneity inherent in real-world fog computing environments.

3 Methodology

3.1 Proposed Approach

We propose a collaborative fault-tolerant model for fog computing by employing autonomous fog nodes that cooperate with each other to maintain the business continuity when failures are predicted or detected. The system is laid out in a multi-tier edge-fog-cloud hierarchical structure wherein the fog nodes act as the main execution layer and collaborate only when necessary. The collaboration happens in a decentralized manner via voluntary interactions between the independently administered fog nodes, which enables both proactive fault tolerance through early migration and reactive fault tolerance whenever faults occur unexpectedly.

The model consists of two core mechanisms that make such collaborative behavior possible. First, a gossip-based state dissemination protocol which allows each fog node to gradually build a complete and consistent view of the global resource state. Done by exchanging periodic summaries of the CPU, memory, and bandwidth loads across a lightweight logical topology that helps fog nodes gain the situational awareness without broadcasting costly global messages. This state need not be perfectly synchronized; rather, it serves as an adequate foundation for the decentralized

decision-making during fault recovery. Second, a reverse-auction mechanism which lets the individual fog nodes negotiate the container placements in a scenario where local recovery is not possible . A node experiencing a predicted or actual fault initiates an auction where candidate fog nodes respond with bids reflecting their capacity and the estimated migration cost. The collaborative fault tolerance is made possible because the nodes voluntarily participate, compete, and are compensated for accepting foreign containers, unlike forced or centrally dictated offloading.

There are three stages to the fault tolerance module: (i) Intra-fog migration attempts to migrate containers within the same fog node whenever a healthy server is available (ii) if local recovery is not feasible, inter-fog migration is triggered, where the candidate fog nodes are evaluated using the aforementioned gossip-informed scoring and reverse-auction bidding (iii) In the event that absolutely no suitable collaborator exists, the task is escalated to the cloud as a final fallback.

The model executes tasks in a lightweight, migratable container format, which allows seamless pausing, checkpointing, transferring, and resumption across the servers or fog nodes. This guarantees that when a fault is predicted, all containers on the affected server are immediately checkpointed and relocated. In case of unexpected faults, the resulting behavior depends on the fault type, like CPU and bandwidth degradations still permit checkpointing, whereas crashes cause complete loss of progress, requiring immediate restart elsewhere [**?**].

Therefore, by combining container-based execution with gossip-driven distributed awareness and a reverse-auction based negotiation, the proposed approach provides a decentralized, prediction-aware, and economically aligned FT architecture which is suitable for heterogeneous, multi-owner fog environments.

3.2 System Model

As discussed previously. The architecture consists of three layers formed by: the edge devices, fog nodes, and a cloud layer that acts as a global fallback. Each fog consists of multiple heterogenous servers which are managed by a local control layer and an FT predictor module. The fog layer creates the main execution foundation and processes the tasks originating from stationary edge devices. These devices constantly generate indivisible tasks that must run to completion and therefore cannot be divided or distributed across servers. Each task is executed inside a dedicated container, and each server can host a multitude of such containers concurrently, constrained only by its CPU, RAM, and bandwidth capabilities. A container can be paused, checkpointed, migrated, and resumed across any fog server or cloud resource, allowing mobility in a response to failures [**?**].

The system is modelled on the assumption of a multi-owner fog environment, where each fog node is being controlled by an independent administrator. A Node may strategically misreport its internal metrics,such as CPU load, memory usage, bandwidth availability, or expected migration time. However, they cannot fake the observable outcomes, including the actual migration duration, SLA adherence, or the execution-time overload. This distinction is vital as misreporting affects negotiation, but actual behavior determines the nodes' accountability. The framework embeds this assumption directly into its FT logic rather than treating it as an external constraint.

Tasks arriving at a particular fog node are encapsulated in a new container and placed on the best available server via that node's own local scheduling policy. The proposed FT framework does not modify this placement algorithm; instead, it comes into play only when faults are predicted or have occurred. The FT predictor is assumed to guarantee a perfect prediction, which means no false positives or false negatives. Thus, every predicted fault will definitely occur, and any server not flagged by the predictor is considered safe until a new prediction arises.

We consider three types of faults: CPU degradation, bandwidth degradation, and server crash. While these faults vary in their operational impact, the FT model treats them equally in terms of triggering the migration, except for one distinction: a crash renders the servers completely inaccessible, causing absolute loss of container progress and requiring restarting, whereas degradations allow checkpointing before migration. The fog-to-cloud communication path is set to have significantly higher latency and lower bandwidth than inter-fog communication links, reinforcing the preference for local or inter-fog recovery before escalating and falling back to the cloud.

This system model enables a decentralized interaction between the fog nodes while preserving each operator's autonomy and supporting truthful or untruthful behavior within defined limits, and leverages containerization to provide strong fault-migration guarantees across the hierarchy.

3.3 Gossip Protocol

As the proposed collaborative FT system does not rely on any central coordinator for maintaining global visibility we adopt a lightweight and scalable mechanism for disseminating resource availability information among fog nodes. The framework uses an eventual consistency gossip protocol that lets each fog node keep an approximate, but still fairly

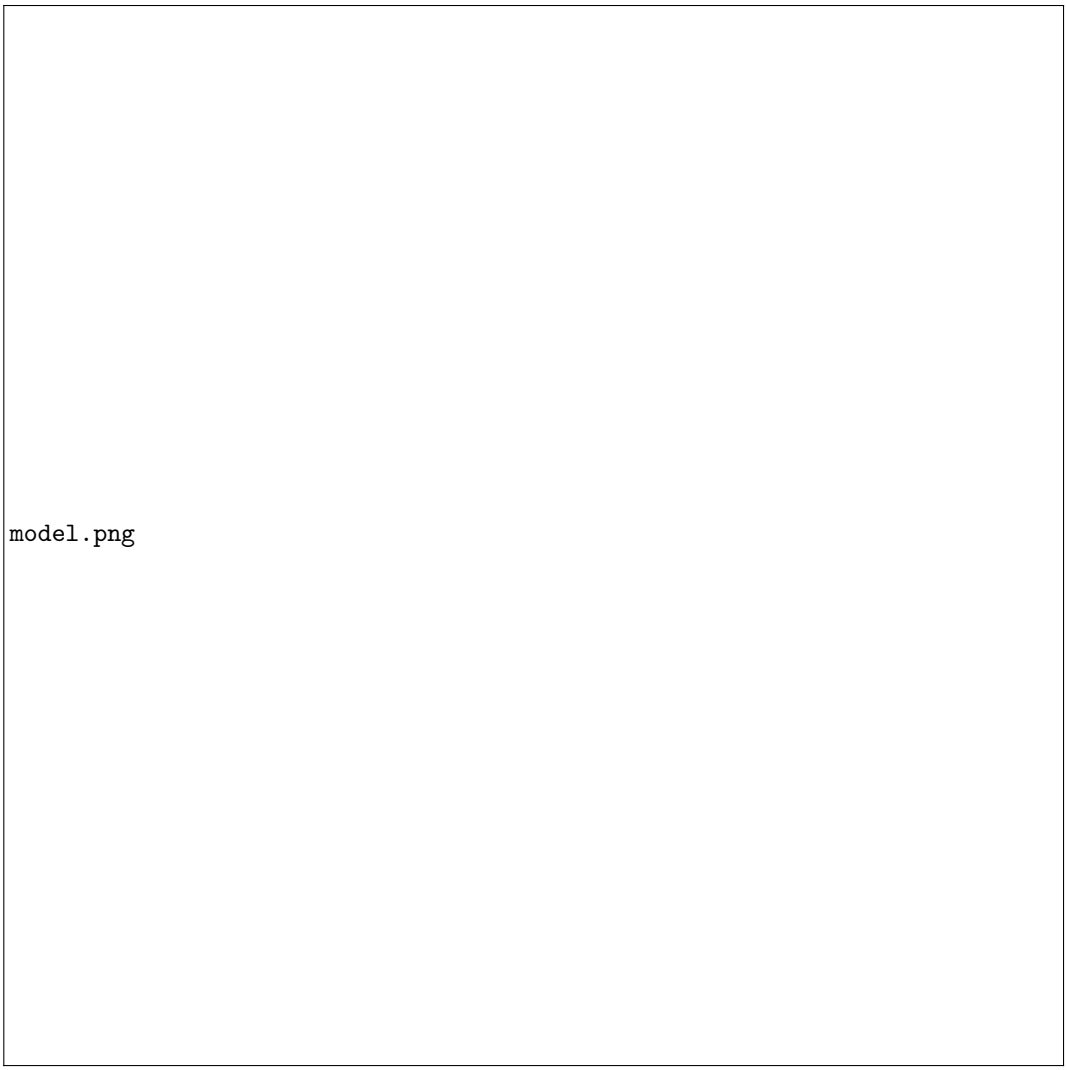

Figure 1: Proposed system model architecture

current view of the system wide resource availability without relying on full broadcast. As the environment involves multi owner fog nodes, this mechanism allows the information exchange to stay light and decentralized, while still providing enough shared visibility for early FT decisions. This eventually consistent state information also allows the fog nodes to make earlier decisions, since even an approximate and slightly stale view can still guide quick migration choices before a fault fully materializes.

In this mechanism every node maintains a table with its knowledge of the whole system's state, called $StateTable_i$. To keep the network usage to a minimum we adopt a ring topology for the gossip dissemination through the system as shown in Fig. **??**. In this topology, every fog node has a designated fog node that it receives an update from and a designated fog node that it sends the update to. Each fog node periodically snapshots its internal load conditions using the normalized CPU, memory, and bandwidth loads given by the below equations:

$$L^i_{cpu} = \frac{\sum_{s \in S_i} U^s_{cpu}}{\sum_{s \in S_i} C^s_{cpu}} \tag{1}$$

$$L^i_{mem} = \frac{\sum_{s \in S_i} U^s_{mem}}{\sum_{s \in S_i} C^s_{mem}} \tag{2}$$

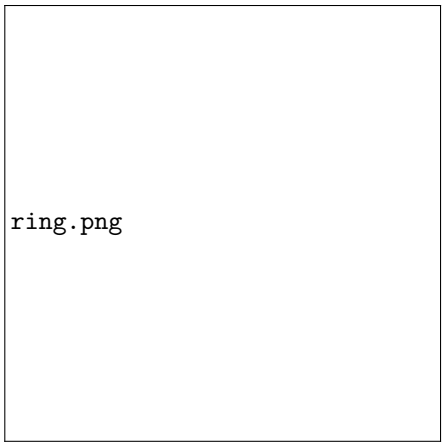

Figure 2: Ring topology between the fog nodes for gossip

$$L_{bw}^{i} = \frac{\sum_{s \in S_i} U_{bw}^{s}}{\sum_{s \in S_i} C_{bw}^{s}} \tag{3}$$

These load values are then attached with a timestamp and updated in its $StateTable_i$, then this updated $StateTable$ is forwarded to its designated upstream ring neighbor. Whenever a fog node receives a $StateTable$ update from its designated downstream ring neighbor, it updates its local $StateTable_i$ with all the updated states. Algorithm **??** presents the complete gossip dissemination process executed periodically at each fog node.

Since fog nodes belong to independent owners, the proposed system assumes that nodes can misreport their loads during gossip exchanges. A node may share lower load values to look more available and attract incoming containers, and this is treated as a normal possibility in such an environment. The architecture is designed so that this kind of misreporting becomes noticeable later through the trust scoring process, where SLA results and actual migration behavior expose the gaps between what was claimed and what actually happened. The proposed approach also assumes that the gossip mechanism is implemented in a simple way where nodes only update their own local values when forwarding the table to the next node, which keeps the process predictable in a multi owner setting. In this way, the gossip process works as the decentralized base that the proposed collaborative FT system relies on. It does not force honesty, but it provides enough shared visibility for the reverse auction and the trust scoring parts to work properly.

3.4 Reverse Auction for Collaborative Migration

The proposed approach adopts a reverse auction, similar to [**?**, **?**], to handle inter fog migration in a way that fits the proposed collaborative system model. Since no node can be forced to take additional load, the system needs a negotiation process rather than a central assignment or distributed forced migration similar to [**?**, **?**, **?**], especially in a multi owner environment where each node follows its own local preferences and constraints. The reverse auction provides a mechanism which allows the candidate nodes to state their willingness to host an incoming container and describe the cost they expect if they take it. This creates a collaborative environment where nodes offer bids that reflect their current conditions, and the originating node can pick a destination that seems reasonable given the information it receives.

When a predicted or detected fault cannot be recovered by local migration on a fog node, the originating fog node issues a migration request to selected candidates determined via gossip-based scoring, discussed later. When a candidate fog node receives a migration request from the originating fog node, it evaluates the request based on the migration time and the impact that it expects on itself. This is formulated as the Bid_{value} that this fog node sends to the originating node. This Bid_{value} is essentially the cost that the fog node at the receiving end of the migration will incur, and is calculated as:

$$Bid_{value} = T_{mig}^{claimed} + k \cdot Impact \tag{4}$$

Algorithm 1 Gossip State Dissemination at Fog Node i

Input: Set of servers S_i, state table $StateTable_i$, neighbor j (upstream), current time t
Output: Updated state tables at neighboring fog nodes

// Compute local load snapshot
$activeServers \leftarrow 0$
$L^i_{cpu} \leftarrow 0$, $L^i_{mem} \leftarrow 0$, $L^i_{bw} \leftarrow 0$
for each server $s \in S_i$ **do**
  if s is not faulted and not predicted to fail **then**
    $L^i_{cpu} \leftarrow L^i_{cpu} + \frac{U^s_{cpu}}{C^s_{cpu}}$, $L^i_{mem} \leftarrow L^i_{mem} + \frac{U^s_{mem}}{C^s_{mem}}$, $L^i_{bw} \leftarrow L^i_{bw} + \frac{U^s_{bw}}{C^s_{bw}}$
    $activeServers \leftarrow activeServers + 1$
  end if
end for
if $activeServers > 0$ **then**
  $L^i_{cpu} \leftarrow L^i_{cpu}/activeServers$, $L^i_{mem} \leftarrow L^i_{mem}/activeServers$, $L^i_{bw} \leftarrow L^i_{bw}/activeServers$
else
  $L^i_{cpu} \leftarrow 1$, $L^i_{mem} \leftarrow 1$, $L^i_{bw} \leftarrow 1$
end if

// Create local state entry with timestamp
$entry_i \leftarrow$ GossipStateEntry$(L^i_{cpu}, L^i_{mem}, L^i_{bw}, t)$
$StateTable_i[i] \leftarrow entry_i$

// Propagate state table to ring neighbors
Send $StateTable_i$ to fog node j

// Upon receiving state table $StateTable_k$ from fog node k (downstream):
for each entry $(nodeId, state) \in StateTable_k$ **do**
  if $nodeId \notin StateTable_i$ or $state.timestamp > StateTable_i[nodeId].timestamp$ **then**
    $StateTable_i[nodeId] \leftarrow state$
  end if
end for

Here $T^{claimed}_{mig}$ is the total migration time that the node expects based on its expectation of bandwidth it can provide, $Claimed_{bw}$, the time to pause the container at the originating node, t_{pause}, and the time to resume the container at the destination, t_{resume}.

$$T^{claimed}_{mig} = t_{pause} + \frac{S_{container}}{Claimed_{bw}/8} + t_{resume} \tag{5}$$

The second component of the bid measures the impact the node might experience by hosting the container based on the below equation:

$$Impact = \frac{R_{cpu}}{C_{cpu}} + \frac{R_{mem}}{C_{mem}} + \frac{R_{bw}}{Claimed_{bw}} \tag{6}$$

This formulation allows the migration overhead and the local resource load to be represented in one simple cost value, so a fog node that is already under heavy load will produce a higher bid and signal that it should not take any more load. Algorithm **??** shows how each candidate fog node builds its bid based on the formulation discussed. After all the individual bids are received, the origin fog node needs to evaluate these bids. During this evaluation the node also incorporates the trust value, from the trust mechanism discussed in next section, within the bids to handle cases where a node may not report its status honestly, either in the bid or the gossip. This results in the effective bid to be evaluated as:

$$Bid_{eff}(i) = \frac{Bid_{value}(i)}{\max(\varepsilon, t(i))} \tag{7}$$

where $t(i)$ is the trust score for node i, discussed in detail in next section. This simple formulation makes nodes with lower trust look more expensive, so they are less likely to be selected even if their raw bid is low. Over time this pushes dishonest behaviour out of the system because nodes that keep misreporting slowly lose competitiveness, while reliable

ones become the natural winners. The origin fog node always selects the bidding node with the lowest Bid_{eff}. The full bid evaluation and winner selection logic at the origin fog node is given in Algorithm **??**.

Algorithm 2 Bid Calculation at Candidate Fog Node i

Input: Migration request for container c from origin fog node j
Output: Bid response containing feasibility and bid value

// Retrieve container requirements
$R_{cpu} \leftarrow c.demandMips$
$R_{mem} \leftarrow c.ramMb$
$R_{bw} \leftarrow c.bandwidth$
$S_{container} \leftarrow c.containerSizeMb$
$D_{sla} \leftarrow c.deadlineSeconds$

// Compute claimed migration time
$Claimed_{bw} \leftarrow$ available bandwidth to fog node j
$T_{transfer} \leftarrow S_{container}/(Claimed_{bw}/8)$
$T_{mig}^{claimed} \leftarrow t_{pause} + T_{transfer} + t_{resume}$

// Compute projected resource load
$L_{cpu} \leftarrow$ current CPU load at fog node i
$L_{mem} \leftarrow$ current memory load at fog node i
$L_{bw} \leftarrow$ current bandwidth load at fog node i
$projected_{cpu} \leftarrow L_{cpu} + R_{cpu}/C_{cpu}$
$projected_{mem} \leftarrow L_{mem} + R_{mem}/C_{mem}$
$projected_{bw} \leftarrow L_{bw} + R_{bw}/Claimed_{bw}$

// Check feasibility
$timeToDeadline \leftarrow \max(0, D_{sla} - t)$
$hasHealthyServer \leftarrow$ any server at i is not faulted and not predicted to fail
$feasible \leftarrow hasHealthyServer$ and $projected_{cpu} \leq 1$ and $projected_{mem} \leq 1$
and $projected_{bw} \leq 1$ and $T_{mig}^{claimed} \leq timeToDeadline$

// Compute bid value
$Impact \leftarrow \frac{R_{cpu}}{C_{cpu}} + \frac{R_{mem}}{C_{mem}} + \frac{R_{bw}}{Claimed_{bw}}$
$Bid_{value} \leftarrow T_{mig}^{claimed} + k \cdot Impact$ ▷ k is resource impact weight

return BidResponse$(i, c.id, feasible, Bid_{value}, Claimed_{bw}, T_{mig}^{claimed})$

3.5 Trust Scoring and Reputation Evolution

Because the proposed collaborative FT mechanism assumes that fog nodes are independently owned and may behave strategically, i,e. lie about their resources, a robust trust scoring process is required to maintain truthful behavior across the gossip exchanges and bidding. For this purpose we propose a trust scoring mechanism where every fog node maintains a trust value $t(i) \in [0, 1]$ for every other fog node in the system. This trust score maintains each node's reliability based on past interactions and is updated after every completed task using both the migration and execution results. This mechanism is essential in enabling collaboration without imposing a central coordinator.

The trust mechanism evaluates every winning node's honesty by comparing the *claimed* versus *actual* migration performance. The *claimed* bandwidth, $Claimed_{bw}$, is shared by the bidding node in the bid response, and the actual bandwidth, $Actual_{bw}$, can be calculated from the observed transfer behavior using:

$$Actual_{bw} = \frac{S_{container}}{T_{transfer}} \times 8 \tag{8}$$

and thus the relative bandwidth honesty error, e_{bw}, can be determined by (ε is to avoid division by 0):

Algorithm 3 Bid Evaluation and Winner Selection at Origin Fog Node

Input: Container c, set of bid responses $Bids$, trust scores T, trust threshold θ_{trust}
Output: Selected winner fog node or cloud fallback

$FeasibleBids \leftarrow \emptyset$
for each bid $b \in Bids$ **do**
  $bidderId \leftarrow b.bidderId$
  $t(bidderId) \leftarrow T[bidderId]$

  // Filter by trust threshold
  if $t(bidderId) < \theta_{trust}$ **then**
    continue ▷ Exclude low-trust nodes
  end if

  // Verify feasibility using local knowledge
  if not $b.feasible$ **then**
    continue
  end if

  // Compute effective bid using trust adjustment
  $Bid_{eff}(bidderId) \leftarrow \frac{b.bidValue}{\max(\varepsilon, t(bidderId))}$ ▷ $t(i)$ is the trust value of node i

  // Compute projected headroom for tie-breaking
  $headroom(bidderId) \leftarrow (1 - projected_{cpu}) + (1 - projected_{mem}) + (1 - projected_{bw})$

  $FeasibleBids \leftarrow FeasibleBids \cup \{(bidderId, Bid_{eff}, headroom, latency)\}$
end for

// Select winner using lexicographic ordering
if $FeasibleBids \neq \emptyset$ **then**
  Sort $FeasibleBids$ by (Bid_{eff} ascending, $headroom$ descending, $latency$ ascending)
  $winner \leftarrow$ first element in sorted $FeasibleBids$
  return $winner.bidderId$
else
  return cloud fallback
end if

$$e_{bw} = \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})} \tag{9}$$

Similarly, the discrepancy between claimed migration time , $T_{mig}^{claimed}$, and actual migration times T_{mig}^{actual} can be evaluated through:

$$T_{mig}^{actual} = t_{pause} + T_{transfer} + t_{resume} \tag{10}$$

where $T_{transfer}$ is the actual transfer time observed by the origin node. The relative migration time honesty error, e_{mig}, can be determined by:

$$e_{mig} = \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})} \tag{11}$$

We define the parameters τ_{bw} and τ_{mig} that represent the dishonesty thresholds for bandwidth and migration time, respectively. Error values above these thresholds indicate intentional misreporting. In case misreporting of migration time and bandwidth is observed, i.e. $e_{bw} > \tau_{bw}$ or $e_{mig} > \tau_{mig}$, the trust update applies a multiplicative decay to the trust score using:

$$t'(i) = t(i) \cdot \delta_{decay} \tag{12}$$

where δ_{decay} is a configurable trust decay value.

Finally, to detect dishonest execution related metrics such as understated CPU or memory load, the framework evaluates SLA performance using:

$$m_{sla} = \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})} \tag{13}$$

where m_{sla} is the SLA margin for the migrated task and negative margins indicate SLA violations, and $T_{complete}$ is the actual completion time of the task. When $m_{sla} < -\tau_{sla}$, where τ_{sla} defines the tolerance for SLA deviation, the trust score is penalized again using the below equation:

$$t''(i) = \max(0, t'(i) - \beta_{violation}) \tag{14}$$

where, $\beta_{violation}$ quantifies how strongly such violations reduce trust.

On the contrary, the proposed trust model also rewards consistently honest fog nodes through:

$$t''(i) = \min(1, t'(i) + \beta_{success}) \tag{15}$$

whenever all honesty conditions are satisfied. The parameter $\beta_{success}$ governs the incremental trust gained from compliant behavior.

Therefore, the final trust value, in case of lying or honest node, is given as:

$$t(i) = t''(i) \tag{16}$$

which integrates both migration-stage honesty checks and SLA-based execution verification. The two phase update to trust score is because migration-stage dishonesty require a multiplicative decay, whereas SLA violations use an additive penalty since an SLA miss may arise from genuine overload rather than deliberate lying. Separating these updates allows the system to distinguish intentional misreporting from operational performance issues while still reflecting both in the final trust value. The parameters τ_{bw}, τ_{mig}, and τ_{sla} therefore define the boundaries of acceptable error, while $\beta_{violation}$ and $\beta_{success}$ control how penalties and rewards shape long-term trust evolution. Together, these mechanisms ensure that repeated dishonesty, whether through gossip, bidding, or execution, reduces a node's influence in future auctions, while sustained honest behavior is economically reinforced.

In addition to these active updates, trust also improves passively over time so that nodes that have not recently hosted tasks can gradually regain competitiveness. The recovery magnitude is proportional to the remaining distance to perfect trust i.e. 1, and at periodic intervals the system applies

$$t_{passive}(i, t) = \min\left(1,\ t(i) + r_{passive} \cdot (1 - t(i))\right) \tag{17}$$

where $r_{passive}$ is a small recovery rate applied at regular time intervals. This slow rise prevents long-term exclusion of inactive nodes while ensuring that previously dishonest nodes do not regain trust abruptly. The complete trust update mechanism, integrating both active evaluation and passive recovery, is formalized in Algorithm **??**.

3.6 Migration Workflow

The proposed migration workflow provides a structured way for the system to decide where a container should move when its current execution can no longer continue due to a predicted fault. The workflow integrates the gossip protocol, reverse-auction mechanism, and trust scoring discussed so far into one decision path that the system follows whenever a migration is required. It defines a three-phase approach that first attempts local recovery, then moves to collaborative inter-fog migration, and finally uses the cloud only when no other option is feasible. Algorithm **??** outlines the full decision process. When the fault predictor identifies that a server is approaching failure, all containers running on that server are paused, checkpointed if possible and removed from that server. Once containers are removed from the failing server, the system initiates the migration workflow.

3.6.1 Phase 1: Intra-fog Recovery

The workflow first attempts to keep the container within the same fog node. In order to do that, the local scheduler checks whether another healthy server within the fog node is available to host the container. If such a server exists, the container resumes execution locally and the migration completes within the fog node. If no server can accommodate the container, the FT system escalates to the next phase and initiates collaborative evaluation across other fog nodes.

Algorithm 4 Trust Evaluation for Node i and Passive Recovery

Input: Fog node i, container c, actual metrics $(Actual_{bw}, T_{mig}^{actual})$, claimed metrics $(Claimed_{bw}, T_{mig}^{claimed})$, completion time $T_{complete}$
Output: Updated trust score $t(i)$

```
// Phase 1: Migration and bandwidth honesty evaluation
e_bw ← |Claimed_bw − Actual_bw| / max(ε, Claimed_bw)
e_mig ← |T_mig^claimed − T_mig^actual| / max(ε, T_mig^claimed)

t'(i) ← t(i)                                   ▷ Initialize with current trust
if e_bw > τ_bw or e_mig > τ_mig then
    t'(i) ← t(i) · δ_decay                     ▷ Apply multiplicative decay
end if

// Phase 2: SLA-based evaluation
m_sla ← (D_sla − T_complete) / max(ε, D_sla)

if m_sla < −τ_sla then
    t''(i) ← max(0, t'(i) − β_violation)       ▷ Apply additive penalty
else
    if e_bw ≤ τ_bw and e_mig ≤ τ_mig then
        t''(i) ← min(1, t'(i) + β_success)     ▷ Apply additive reward
    else
        t''(i) ← t'(i)                         ▷ No SLA reward if migration dishonest
    end if
end if

t(i) ← t''(i)                                  ▷ Store updated trust

// Passive recovery (applied periodically for all fog nodes)
procedure PASSIVERECOVERY(i, t_current, t_last_recovery)
    if t_current − t_last_recovery ≥ Δt_recovery then
        increment ← r_passive · (1 − t(i))
        t(i) ← min(1, t(i) + increment)
        Update t_last_recovery(i) ← t_current
    end if
end procedure

return t(i)
```

3.6.2 Phase 2: Inter-fog migration via reverse auction

When the container cannot be migrated within the same fog node, the FT system attempts collaborative FT. In this phase the workflow evaluates the collaborating fog nodes using the state information that was exchanged through the gossip mechanism. First the nodes with stale state entries are filtered out. Then for every remaining candidate node, the FT system computes the workload proportions based on the container being migrated,

$$p_{cpu}, \qquad p_{mem}, \qquad p_{bw}, \tag{18}$$

where p_{cpu}, p_{mem}, and p_{bw} represent how much of the container's demand is CPU-intensive, memory-intensive, or bandwidth-intensive. These proportions describe the container's dominant resource profile and always sum to one.

The system then determines the task urgency through the normalized slack factor,

$$U_{norm} = 1 - \frac{\max(0,\ D_{sla} - T_{pred})}{D_{sla}}, \tag{19}$$

where D_{sla} is the task's deadline and T_{pred} is the predicted remaining execution time. Larger U_{norm} values indicate that the deadline is approaching and the task becomes more sensitive to delays.

These values produce the dynamic scoring weights. The bandwidth, CPU and memory weight are given by

$$\gamma = p_{bw} + U_{norm}(1 - p_{bw}), \tag{20}$$

$$\alpha = (1-\gamma)\frac{p_{cpu}}{p_{cpu} + p_{mem}}, \qquad \beta = (1-\gamma)\frac{p_{mem}}{p_{cpu} + p_{mem}}, \tag{21}$$

where γ increases when the workload is bandwidth-heavy or when urgency is high, and α and β split the remaining weight proportionally based on the CPU versus memory composition of the container.

Each candidate fog node i is then assigned a dynamic score based on Eq. **??**:

$$Score(i) = \alpha\left(1 - L^i_{cpu}\right) + \beta\left(1 - L^i_{mem}\right) + \gamma\left(1 - L^i_{bw}\right), \tag{22}$$

where L^i_{cpu}, L^i_{mem}, and L^i_{bw} are the normalized CPU, memory, and bandwidth loads of node i taken from gossip. The $(1 - L)$ terms indicate the available remaining capacity in each resource dimension.The complete scoring procedure is formalized in Algorithm **??**

Algorithm 5 Dynamic Scoring of Fog Nodes for Container Migration

Input: Container c, state table $StateTable$, staleness threshold θ_{stale}, current time t
Output: Ranked list of candidate fog nodes

// Compute workload proportions
$total \leftarrow R_{cpu} + R_{mem} + R_{bw}$
$p_{cpu} \leftarrow R_{cpu}/total$
$p_{mem} \leftarrow R_{mem}/total$
$p_{bw} \leftarrow R_{bw}/total$

// Compute normalized urgency
$slack \leftarrow \max(\varepsilon, D_{sla} - t)$
$U \leftarrow 1/slack$
$U_{norm} \leftarrow \min(1.0, U \cdot k)$ ▷ $k = 0.1$ is urgency scaling factor

// Compute dynamic weights
$\gamma \leftarrow p_{bw} + U_{norm}(1 - p_{bw})$
$W_{remaining} \leftarrow 1 - \gamma$
$\alpha \leftarrow W_{remaining} \cdot \frac{p_{cpu}}{p_{cpu}+p_{mem}}$
$\beta \leftarrow W_{remaining} \cdot \frac{p_{mem}}{p_{cpu}+p_{mem}}$

// Score all non-stale fog nodes
$Candidates \leftarrow \emptyset$
for each fog node i in $StateTable$ **do**
    if i is local node **then**
        continue
    end if
    $entry \leftarrow StateTable[i]$
    if $t - entry.timestamp > \theta_{stale}$ **then**
        continue ▷ Discard stale entries
    end if
    $Score(i) \leftarrow \alpha(1 - L^i_{cpu}) + \beta(1 - L^i_{mem}) + \gamma(1 - L^i_{bw})$
    $Candidates \leftarrow Candidates \cup \{(i, Score(i))\}$
end for

// Sort by score descending
Sort $Candidates$ by score in descending order
return $Candidates$

Candidates that achieve high scores proceed to the negotiation stage. The FT system then initiates a reverse auction, with these candidate nodes, where the origin fog node transmits the container details to the candidate nodes. Each

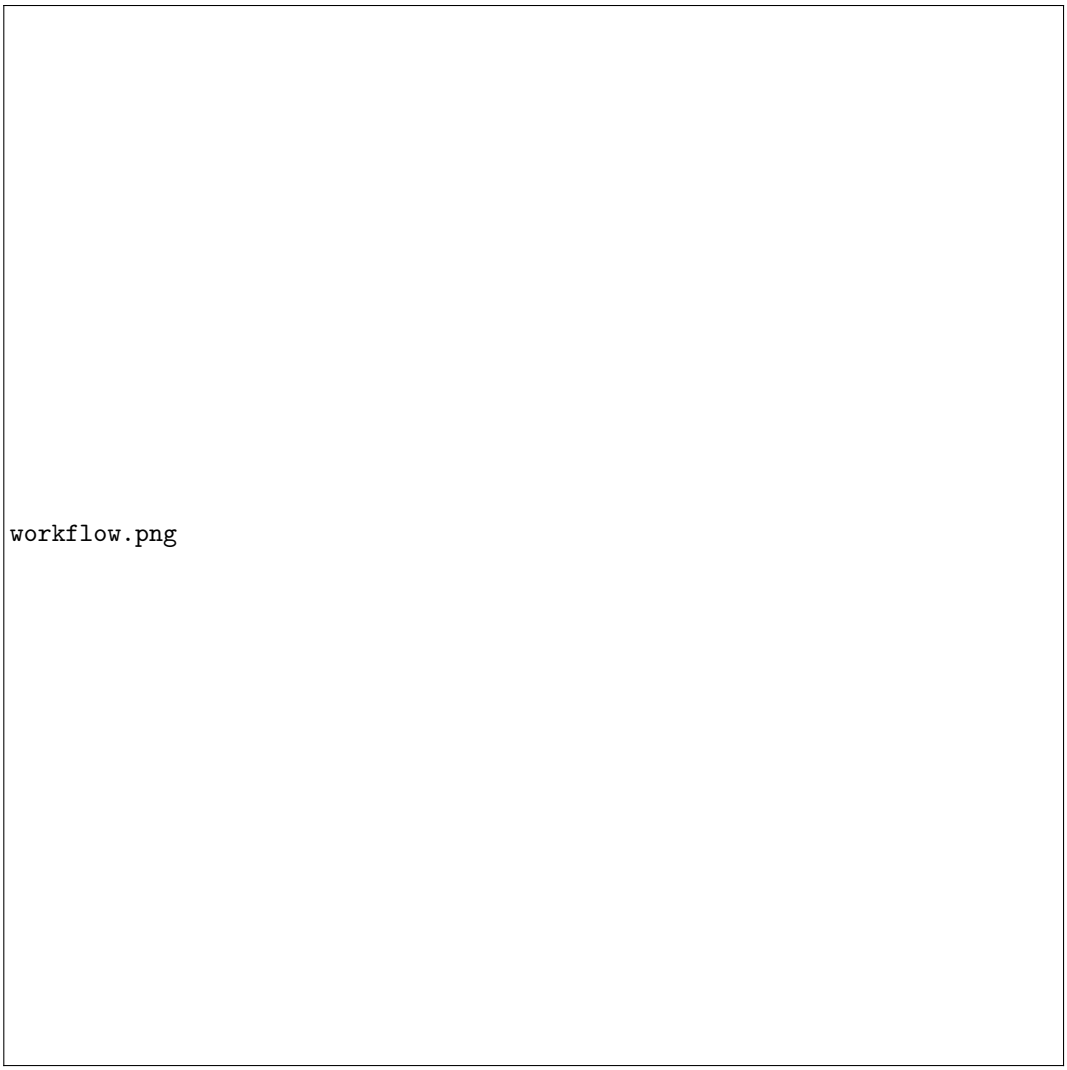

Figure 3: FT Migration Workflow

candidate fog node submits a bid calculated using Eq. **??** and sends them to the origin node as the bid response (which also includes other information). The origin node then transforms these submitted bids into effective bids by applying the trust adjustment rule according to Eq. **??**, and the fog node with lowest effective bid is selected (As this is a reverse auction). The container is then migrated to the selected fog node, and the observed transfer time T_{transfer} is recorded. On arrival, the receiving fog node places the container on one of its healthy servers using its internal scheduler.

If no collaborating fog node is able to host the container, the workflow moves to the cloud fallback phase.

3.6.3 Phase 3: Cloud Fallback

When neither local nor inter-fog recovery is possible, the workflow sends the container to the cloud as the final fallback option. The cloud does not participate in the bidding process. The container is transferred, T_{transfer} is recorded, and the container resumes or restarts depending on whether a checkpoint was available.

Overall, the workflow creates a structured and flexible response to failures. It prioritizes local recovery, leverages distributed scoring and auction-based cooperation when needed, and uses the cloud only as a final fallback. This layered approach maintains task continuity across both predicted and unexpected fault scenarios, even in environments where fog nodes might misreport their state or behave strategically.

Algorithm 6 Collaborative Migration Workflow at Fog Node i

Input: Container c, fault event f, servers S_i, state table $StateTable_i$
Output: Container migrated to new location

// Handle fault and checkpoint if possible
$sourceServer \leftarrow$ server hosting container c
if f is irrecoverable **then**
    Remove c from $sourceServer$ without checkpointing
    $c.resetProgress()$ ▷ Work progress lost, restart required
else
    Pause container c
    Checkpoint c with current progress at time t ▷ Fault is recoverable
    Remove c from $sourceServer$
end if

// Phase 1: Attempt intra-fog migration
$localTarget \leftarrow$ default scheduler places c on a healthy server in S_i
if $localTarget \neq$ null **then**
    Resume execution of c
    Record intra-fog migration
    return
end if

// Phase 2: Inter-fog migration via reverse auction
$Candidates \leftarrow$ Score all fog nodes using Algorithm **??**
Limit $Candidates$ to top N_{max} nodes

// Request bids from candidates
$Bids \leftarrow \emptyset$
for each fog node $j \in Candidates$ **do**
    Send migration request to j
    Await bid response from j (computed via Algorithm **??**)
    $Bids \leftarrow Bids \cup \{$ received bid$\}$
end for

// Select winner using trust-adjusted bidding
$winner \leftarrow$ SelectWinner$(c, Bids)$ using Algorithm **??**
if $winner \neq$ null and $winner \neq$ cloud **then**
    Initiate transfer of c to fog node $winner$
    Measure actual bandwidth $Actual_{bw}$ and migration time T_{mig}^{actual}
    Winner's default scheduler places c on appropriate server
    Record inter-fog migration
    return
end if

// Phase 3: Cloud fallback
if cloud is available **then**
    Initiate transfer of c to cloud
    Cloud's default scheduler places and resumes/restarts c
    Record cloud migration
else
    Enqueue c for retry when resources become available
end if

3.6.4 Payment Settlement

Once the migrated task completes, the node that won the auction and executed the container informs the origin node about the completion. The origin fog node on receiving this alert evaluates the final payment by applying the final payment rule defined in Algorithm **??**, and carries out the payment to the winning node.

Algorithm 7 Payment Settlement and Combined Trust Update

Input: Container c, winning fog node i, completion time $T_{complete}$, bid context
Output: Payment transfer and updated trust score

// Retrieve bid and actual metrics
$Bid_{value} \leftarrow$ original bid value from fog node i
$Claimed_{bw} \leftarrow$ bandwidth claimed in bid
$T_{mig}^{claimed} \leftarrow$ migration time claimed in bid
$Actual_{bw} \leftarrow$ measured actual bandwidth during transfer
$T_{mig}^{actual} \leftarrow$ measured actual migration time

// Compute error metrics
$e_{bw} \leftarrow \frac{|Claimed_{bw} - Actual_{bw}|}{\max(\varepsilon, Claimed_{bw})}$, $e_{mig} \leftarrow \frac{|T_{mig}^{claimed} - T_{mig}^{actual}|}{\max(\varepsilon, T_{mig}^{claimed})}$

// Compute SLA margin
$m_{sla} \leftarrow \frac{D_{sla} - T_{complete}}{\max(\varepsilon, D_{sla})}$

*// Update trust using combined evaluation (Algorithm **??**)*
$t_{updated}(i) \leftarrow$ TrustUpdate$(i, c, Actual_{bw}, T_{mig}^{actual}, Claimed_{bw}, T_{mig}^{claimed}, T_{complete})$

// Calculate payment with penalties
$P_{lie} \leftarrow \lambda(e_{bw} + e_{mig})$, $P_{sla} \leftarrow \mu \cdot \max(0, -m_{sla})$

$Payment \leftarrow \max(0, Bid_{value} - P_{lie} - P_{sla})$

// Execute payment transfer
if fog node $i \neq$ owner of c and $i \neq$ cloud **then**
    Debit $Payment$ from owner's account
    Credit $Payment$ to fog node i's account
end if

Record settlement metrics: $(c.id, i, Bid_{value}, Payment, t_{updated}(i), m_{sla})$

The final payment to the winner node is carried out with a penalty attached to it for missing the SLA or reporting untruthful capacities. The final payment amount is determined using the below equation:

$$\text{Payment} = \max\left(0,\ Bid_{Value} - P_{\text{lie}} - P_{\text{sla}}\right), \tag{23}$$

where the P_{lie} is the dishonesty penalty for lying about the bandwidth and migration time and P_{sla} is the penalty for SLA violation.

The dishonesty penalty is calculated as

$$P_{\text{lie}} = \lambda\left(e_{bw} + e_{mig}\right), \tag{24}$$

where e_{bw} and e_{mig} are derived from trust scoring using Eq. **??** and Eq. **??** and λ controls the severity of this penalty.

The SLA penalty is calculated as

$$P_{\text{sla}} = \mu \cdot \max\left(0,\ -m_{sla}\right). \tag{25}$$

Here m_{sla} is the SLA margin of the completed task from the trust score using Eq. **??**, and μ scales how strongly SLA violations reduce the final payment.

Intra-fog placements always yield zero payment, and cloud placements follow the same rule while not affecting trust values. This settlement step finalizes the workflow by aligning the economic incentives of the hosting fog nodes with the honesty and performance expectations defined earlier.

3.7 Experimental Setup

The evaluation of the proposed collaborative FT approach was conducted using a customized version of iFogSim2[1] that we modified to support all components of our design, including container-level execution, checkpointing, migration,

[1] https://github.com/Cloudslab/iFogSim

gossip-driven state dissemination, urgency-aware scoring, reverse-auction bidding, and trust-based negotiation. These extensions allowed the simulator to faithfully reproduce the behavior of the proposed mechanisms rather than relying on the default iFogSim model. All workloads, failures, and communication events were generated synthetically within the modified simulator, enabling controlled experiments and full reproducibility without reliance on external datasets.

Table **??** summarizes the system-level configuration used in all experiments. The simulated environment consists of ten fog nodes arranged in a logical ring for gossip communication. Each fog node hosts three to five heterogeneous servers with capacities ranging from $C^s_{cpu} \in [2000, 4000]$ MIPS, $C^s_{mem} \in [4096, 8192]$ MB, and $C^s_{bw} \in [3500, 9000]$ Mbps. Unlike uniform configurations, the topology employs heterogeneous capacity multipliers ranging from 1.0 to 1.2 across fog nodes, creating realistic resource imbalances. Each fog node is connected to a variable number of stationary edge devices, ranging from 2 to 10 devices per node, that continuously generate tasks. Cloud resources were provisioned with significantly higher capacity to serve as a reliable fallback destination. Network delays follow the fog-computing hierarchy, where edge-fog links are fastest, inter-fog communication is moderately fast, and fog-cloud links are slower and more bandwidth-constrained.

Table **??** summarizes the faults, workload parameters, bidding configuration, trust parameters, and SLA specifications used in the experiment. Faults are generated according to a Poisson distribution. The faults are injected and arrive with a mean inter-failure time of 120 seconds, and each fault includes a prediction window of 120 seconds and a recovery time of 120 seconds. Three types of faults are considered namely: CPU degradation, bandwidth degradation, and server crash, which follow the probabilities listed in the Table **??**. CPU and bandwidth degradations allow the system to checkpoint containers before migrating them, while server crashes cause complete loss of progress. During inter-fog migration, fog nodes respond with bids computed using the claimed migration time and resource impact formulations defined in Equations **??** and **??**, parameterized by the constants k, t_{pause}, and t_{resume}. Trust scores evolve according to measured discrepancies in bandwidth and migration honesty, SLA behavior, and passive recovery, governed by the thresholds τ_{bw}, τ_{mig}, and τ_{sla}.

Task workloads were selected to produce realistic load conditions across the fog layer. Each edge device generates 32 tasks that arrive at random times during the simulation, with a configurable jitter. Resource requirements for tasks including CPU, memory, bandwidth, runtime, and container size, are sampled from the value ranges shown in Table **??**. SLA deadlines scale with task runtime according to the multiplier provided in the configuration.

A baseline system was implemented for comparison, identical to the proposed approach except that gossip dissemination, trust scoring, and auction-based collaboration were disabled. Instead, all migration decisions were handled by a centralized scheduler. The centralized scheduler has realtime global view of all servers capacities and it can force fog nodes to take on containers that it wants to migrate. This allows direct comparison between our decentralized collaborative FT approach and a simplified centralized system under identical workload and fault conditions.

3.8 Metrics for Comparison

To better understand how the proposed collaborative FT model behaves compared to the baseline configurations, we rely on a set of metrics which capture how well the system maintains its timeliness, how smoothly it handles migrations, and how stable it remains when a fault occurs. These metrics highlight the practical influence of gossip visibility, trust-aware bidding, and a prediction driven migration. They help in showing how these components shape the flow of tasks through the fog layer.

SLA violation rate Each task in the system has a particular completion deadline, and a violation is recorded whenever the task fails to finish within its deadline. Since both predicted faults and unexpected degradations can delay execution or increase migration overhead, the SLA violation rate provides a straightforward representation of whether the proposed FT framework improves reliability under given fault conditions or not. A lower violation rate generally reflects a more stable execution.

Average migration time This measures how long a given container remains in the migration process, it starts the moment the container and its processes have been paused up until the point when execution resumes at the destination. A container may experience several migrations during its lifetime, especially in environments that have repeated or overlapping faults, and for this reason, the average migration time becomes an essential metric to monitor. It shows whether the FT mechanisms keep recovery overhead manageable or if the repeated migrations accumulate into noticeable performance penalties. Lower values usually indicate smoother and more predictable recovery.

Average makespan time Makespan refers to the total duration a task spends in the system; it includes the execution time, any delays caused by predicted or reactive faults, and all migration-related overhead. We compute this value for each task and then average across the workload. Makespan offers a broad view of end-to-end

Table 1: System and Network Configuration

Parameter	Value
Number of Fog Nodes	10
Servers per Fog Node	3–5
CPU Capacity per Server (C^s_{cpu})	2000–4000 MIPS
Memory per Server (C^s_{mem})	4096–8192 MB
Bandwidth per Server (C^s_{bw})	3500–9000 Mbps
Storage per Server	100000–200000 MB
Uplink/Downlink Bandwidth	6000–9000 Mbps
Capacity Multiplier Range	1.0–1.2
Cloud CPU Capacity	9000 MIPS
Cloud Memory	45000 MB
Cloud Bandwidth	40000 Mbps
Cloud Storage	200000 MB
Cloud Uplink/Downlink	40000 Mbps
Edge Devices per Fog Node	2–10 (variable)
Edge Latency	3 ms
Edge Bandwidth	1500 Mbps
Edge Heterogeneity Jitter	10%
Inter-Fog Latency	8 ms
Inter-Fog Bandwidth	1500 Mbps
Fog-Cloud Latency	70 ms
Fog-Cloud Bandwidth	800 Mbps
Gossip Enabled	True
Gossip Interval	7 s
Gossip Staleness Threshold	3 intervals

performance and helps reveal whether tasks are progressing efficiently or not despite the migrations and disruptions.

Total number of migrations We track the total number of migrations and classify them into three categories: intra-fog, inter-fog, and cloud migrations, in order to clearly show how the system responds to faults. Intra-fog migrations reflect the system's ability to recover locally. Inter-fog migrations show how often cooperation among fog nodes is needed. Cloud migrations indicate situations where neither local recovery nor inter-fog support is feasible and act as a global fall back. Observing the distribution of these migration types highlights and helps compare the behavior of the proposed FT approach to its baseline configuration, particularly in terms of how often unnecessary or avoidable migrations occur.

Together, all these metrics provide a thorough view of the responsiveness, efficiency, and stability, thereby allowing us to compare the proposed FT approach with the baseline configuration in a controlled and meaningful manner.

4 Results

This section presents the performance evaluation of the proposed *Collaborative Decentralized (Ours)* framework compared against a *Centralized (Baseline)* approach. The experiments were conducted using the configuration detailed in Section **??**, with metrics collected over 1793 generated tasks, 85 faults , and simulated over a period of 1hr (3600s). The findings focus on task completion latency, system stability, migration behavior, and fault tolerance efficiency.

4.1 Performance Overview

Table **??** summarizes the core performance metrics for both operational modes. The *Collaborative Decentralized (Ours)* approach achieved an average makespan of 184.84 seconds, which is lower than the 188.36 seconds observed in the *Centralized (Baseline)*. Additionally, the load imbalance standard deviation was reduced from 14.24 in the baseline to 11.78 in the proposed approach, indicating a more uniform distribution of workload across the fog layer.

Table 2: Fault, Bidding, Trust, and Task Configuration

Category	Parameter and Value
Fault Model	
Prediction Lead Time	120 s
Mean Time Between Failures	120 s
Fault Start Delay	90 s
Recovery Time	120 s
CPU Failure Probability	0.33
Bandwidth Degradation Probability	0.34
Server Crash Probability	0.33
Bidding Model	
Max Fog Bidders	4
Pause Time (t_{pause})	1.5 s
Resume Time (t_{resume})	1.5 s
Resource Impact Weight (k)	0.6
Bid Timeout	5.0 s
Retry Migration Interval	10.0 s
Max Migration Retries	10
Trust Model	
Trust Enabled	True
Decay Factor (δ_{decay})	0.5
Trust Recovery Factor	0.05
Trust Threshold	0.5
Bandwidth Dishonesty Threshold (τ_{bw})	0.3
Migration Dishonesty Threshold (τ_{mig})	0.5
SLA Violation Threshold (τ_{sla})	0.3
SLA Success Bonus ($\beta_{success}$)	0.02
SLA Violation Penalty ($\beta_{violation}$)	0.10
Payment Dishonesty Penalty (λ)	0.25
Payment SLA Penalty (μ)	0.10
Passive Recovery Rate ($r_{passive}$)	0.01
Passive Recovery Interval	21 s
Task Model	
Tasks per Edge Device	32
Mean Inter-Arrival Time	160 s
Task Generation Cutoff Buffer Ratio	1.6
Runtime Range	240–400 s
Demand MIPS Range	200–600 MIPS
RAM Range	128–512 MB
Bandwidth Range	150–550 Mbps
Container Size Range	200–420 MB
SLA Runtime Multiplier	1.2

Table 3: Overall Performance Comparison

Metric	Collaborative Decentralized (Ours)	Centralized (Baseline)
Average Makespan (s)	184.84	188.36
Load Imbalance (StdDev)	11.78	14.24
Total Migrations	479	580
SLA Violation Ratio	5.18%	5.18%
Total SLA Violations	93	93

4.2 Migration Analysis

A detailed breakdown of migration behavior is presented in Table **??**. The total number of migrations required to maintain system stability was significantly lower in the decentralized mode (479) compared to the centralized baseline (580).

Table 4: Migration Distribution by Type

Migration Type	Collaborative Decentralized (Ours)	Centralized (Baseline)
Intra-fog Migrations	139	128
Inter-fog Migrations	281	421
Cloud Migrations	59	31
Total	**479**	**580**

The breakdown reveals that the proposed approach performed fewer inter-fog migrations (281) than the baseline (421), suggesting that the gossip-informed scoring mechanism effectively identified suitable candidates without necessitating excessive task hops. However, the decentralized approach utilized cloud fallbacks slightly more frequently (59) than the centralized baseline (31), likely due to the lack of a global view forcing a cloud escalation when local neighborhoods were saturated. Conversely, the proposed approach achieved a higher number of intra-fog recoveries (139 vs. 128), prioritizing local resource utilization before attempting remote transfers.

4.3 Reliability and Fault Tolerance

Both systems demonstrated identical reliability in terms of Service Level Agreement (SLA) adherence. As shown in Table **??**, both the Collaborative Decentralized and Centralized modes recorded a violation ratio of approximately 5.18%, with exactly 93 violations out of 1793 tasks.

In terms of fault recovery, both modes successfully recovered 85 faults with an average lead time of 120.0 seconds. The average time taken to complete a migration was 1.27 seconds for the decentralized approach and 1.13 seconds for the centralized baseline. The slightly higher migration time in the decentralized mode can be attributed to the negotiation overhead inherent in the reverse auction process, which is absent in the direct assignment logic of the centralized scheduler.

Table 5: Comparison of Average Migration Time

Mode	Average Migration Time (s)
Collaborative Decentralized (Ours)	1.27
Centralized (Baseline)	1.13

4.4 Network and Economic Metrics

The decentralized nature of the proposed solution introduces network overhead due to state dissemination. The gossip protocol generated 5,140 messages totaling approximately 3.26 MB of data. In contrast, the centralized baseline incurred zero gossip overhead as it relies on a central authority. This overhead is incurred over 1 hour of simulation which minimal, and the delay or transmission interval can be further reduced to optimize the gossip messages generated.

5 Discussion

This work evaluates whether a decentralized, auction-based fault tolerance mechanism could maintain service continuity in a multi-owner fog environment without the latency penalties typically associated with distributed coordination. The results validate the proposed framework, demonstrating that shifting decision-making authority to individual fog nodes achieves a superior balance of autonomy, stability, and efficiency compared to a centralized baseline.

5.1 Performance

A critical finding is that our proposed approach achieved a lower average makespan (184.84 s) than baseline (188.36 s). This challenges the assumption that global optimization always yields the most efficient schedule. In contrast to the centralized hybrid optimization (MHHO-ACO) used in [**?**], our results suggest that centralized solvers can become bottlenecks during high-frequency fault events. By decentralizing the decision process, our framework allows nodes to react immediately to local state changes and autonomously asses their condition and subsequently decide whether or not to partake in the auction.

5.2 Migration Dynamics and Stability

The structural impact of the proposed mechanism is evident in the significant reduction of total migrations (479 vs. 580 for the baseline). Specifically, Inter-fog migrations decreased (281 vs. 421), while Intra-fog migrations increased (139 vs. 128). This shift indicates that the auction mechanism acts as a strict filter, prioritizing local recovery and preventing nodes from accepting tasks they cannot sustain. The reduction in inter-fog migrations suggests that the proposed scoring and bidding mechanisms effectively filter out sub-optimal migration candidates that a central controller might otherwise force a node to accept. This confirms the hypothesis that enabling node autonomy prevents "ping-pong" migration effects, leading to a more stable execution environment.

However, the lack of global visibility resulted in higher cloud fallbacks (59 vs. 31). This was an expected trade-off in decentralized edge systems, as was also observed in the fuzzy-logic migration framework in Pala and Pinarer [**?**], where local uncertainty often necessitates escalation to the cloud.

5.3 Reliability

Our proposed approached maintained an identical SLA violation ratio of 5.18% (93 violations) compared to the baseline, demonstrating that decentralization does not necessarily compromise reliability. However, our decentralized approach utilized cloud fallbacks more frequently, which is an expected trade-off of the "partial view" inherent in the gossip protocol, where a decentralized node may occasionally fail to find a viable neighbour in its gossip table and default to the cloud.

5.4 Latency Trade-offs in Negotiation

The introduction of the reverse-auction mechanism introduces a calculated overhead. As observed in the results, the average migration time for the proposed approach (1.27 s) was slightly higher than the baseline (1.13 s). This additional latency is the cost of establishing trust and fairness in a multi-owner environment. As noted above however, this minor increase in migration latency did not severely impact the overall task completion time, implying that while the act of migrating takes longer due to bidding, the resulting placement is of higher quality. Moreover, as noted in [**?**], stateful migration technologies like CRIU can significantly reduce startup costs, rendering this slight negotiation delay negligible in the context of the overall makespan reduction.

Moreover, as discussed in the results, the gossip protocol introduces slight network overhead which contributes to bandwidth degradation. However, this represents an acceptable trade-off since the impact on overall performance is minimal, and the protocol provides essential state context to the origin fog node. the gossip protocol also prevents network storms by eliminating the need to broadcast bid requests to the entire network.

5.5 Economic Incentives and Load Balancing

The proposed approach demonstrated superior load balancing capabilities, with a load imbalance standard deviation of 11.78 compared to 14.24 in the baseline. This improvement validates the economic incentive model adapted from [**?**]. By dynamically pricing resources based on their scarcity (CPU, RAM, and Bandwidth), the reverse auction naturally guides containers toward the least loaded nodes. Nodes are compensated more when they assist during high-demand periods, aligning individual profit motives with the system-wide goal of load equalization.

Furthermore, the improved load balance indicates that the proposed economic scoring mechanism effectively prevents the “thundering herd” problem common in central scheduling where a central scheduler might overload a single optimal node. This extends the work [**?**] by demonstrating that economic incentives can guide decentralized grouping as effectively as heuristic methods.

5.6 Robustness Against Strategic or Dishonest Behavior

This section discusses how the proposed collaborative FT system behaves in situations when fog nodes attempt to misreport their current state or strategically manipulate the migration process. These scenarios highlight how the proposed mechanisms, gossip protocol, dynamic scoring (Eq. **??**), bid formulation, and the trust evolution equations work together to ensure that FT is carried out in a trustworthy and timely manner. The scenarios also show how dishonest behavior by fog nodes eventually exposes itself once the container completes migration and execution.

Scenario 1: A fog node understates its bid. Imagine a fog node that tries to look attractive by reporting a very small $T_{mig}^{claimed}$ in Eq. **??** and a small impact value in Eq. **??**. The bid it sends to the origin fog node appears to be the best i.e. cheapest, so it may occasionally win. However, once the migration completes, the system recomputes the actual bandwidth using Eq. **??** and the actual migration time using Eq. **??**. If the real conditions are slower than what the node claimed, the error values e_{bw} and e_{mig} from Eq. **??** and Eq. **??** become considerably high, leading to penalized payment using Eq. **??** and trust decay using Eq. **??**. In addition, after the task has completed execution, the node will miss the SLA for the task leading to a negative m_{sla}, which leads to payment penalty from Eq. **??** and trust slashing using Eq. **??**. Since Bid_{eff} in Eq. **??**, evaluated by the origin node, depends on trust, even a few dishonest migrations push the trust value lower and make future effective bids very large. The node that originally looked appealing now becomes too costly for the system to select. As a result, understating a bid only provides a short term benefit, with penalty, before the system naturally moves away from that node.

Scenario 2: A fog node overstates its bid. Now consider a fog node that claims very large transfer times and a large impact value in Eq. **??**, to keep its load lower and get more payment. If that node somehow wins the bid for migration, even then nothing in the migration exposes any dishonesty because the actual measurements still match the claims. However, the inflated values simply make Bid_{eff} in Eq. **??** much higher than the other candidates, making it harder for it to win bids and get payments. It remains honest, so trust does not decrease, but it almost never wins because the bid selection mechanism will not choose the expensive option. In this sense, overstating is also a losing strategy because the system does not need to penalize it, the auction mechanic already takes care of it.

Scenario 3: A fog node lies about bandwidth or migration time. Consider a fog node that tries to bid strategically, by announcing a high $Claimed_{bw}$ in Eq. **??** and a short $T_{mig}^{claimed}$ in Eq. **??**, in order to win the bid and get paid more. The bid appears attractive, and the node may even be selected by the origin fog node. Once the container begins transferring, the origin node recomputes the actual bandwidth through Eq. **??** and the actual migration time using Eq. **??**. If the transfer behaves significantly worse than claimed, the resulting error values e_{bw} and e_{mig} from Eq. **??** and Eq. **??** become high. This immediately introduces a dishonesty penalty through Eq. **??**, reducing the final payment for hosting the container, and also results in trust decay using Eq. **??**. Since Bid_{eff} in Eq. **??** depends on trust, the node's future bids become expensive relative to honest competitors. After only a few such migrations, the node loses its ability to win auctions and is pushed out of the collaboration.

Scenario 4: A fog node lies about CPU or RAM load. A different type of misreporting occurs when a fog node lies specifically in its bid by exaggerating its available CPU or RAM capacity. In this case the gossip values in Eq. **??** through Eq. **??** may still be honest, but the bid claims that hosting the container will have little impact on the node. This lowers the calculated $Impact$ term in Eq. **??** and produces a more attractive Bid_{value} in Eq. **??**. Once the node wins the auction and begins execution, the truth emerges. The overloaded server slows down the task execution, increasing the actual completion time $T_{complete}$ and causing the SLA margin in Eq. **??** to become negative. This triggers the SLA penalty from Eq. **??**, reducing the final payment and also decreasing trust using Eq. **??**. With weakened trust, even correctly reported future bids produce larger Bid_{eff} values through Eq. **??**. As a result, the node becomes less competitive and eventually stops winning auctions. The system does not need to verify CPU or RAM reports explicitly because the execution slowdown reveals the dishonesty, and payment and trust penalties enforce the correction.

Scenario 5: A fog node lies in the gossip state. A fog node may instead choose to manipulate its gossip load in the $StateTable$ updates, reporting lower loads to make sure its selected in the scoring function of Eq. **??**. This can help the node attract more migration bid requests in the short term. However, when the node receives more containers than it can actually handle, the execution time for tasks start to explode leading to delays. These delays lead to the cascading evaluations discussed in Scenario 4 above which ultimately reveal the false claims and penalizes the node making it unattractive in future considerations.

Taken together, these scenarios show that dishonest behavior undermines a fog node's ability to participate in the system. The gossip equations, the scoring formula, the bid formulation, the effective bid equation, and the trust updates all reinforce one another. Honest nodes remain competitive and dishonest ones gradually lose influence. The system maintains stable cooperation even when individual nodes attempt to manipulate their reported conditions.